CuPath: Cuckoo-Filter-Guided Memory Request Transformation for Wafer-Scale GPUs

HPCA 2027 Submission **#NaN**
Confidential Draft
Do NOT Distribute!!

***Abstract*—Wafer-scale GPUs integrate many GPU processing modules (GPMs) and distributed HBM into one logical device through a high-bandwidth, low-latency on-wafer network. Although this organization reduces conventional multi-GPU communication overhead, its memory path remains expensive. A remote cache miss may traverse the requesting-GPM L2, both RDMA endpoints, the wafer network, the home-GPM L2, and HBM. Existing architectures optimize these components independently even though different optimization opportunities emerge along the request path. We propose *CuPath*, which uses typed Cuckoo Filters as a compact control plane for coordinating memory requests along the local and remote paths. The control plane coordinates (2) an RDMA batcher that merges duplicate remote requests and groups requests to the same home GPM and page, (3) a remote-reuse policy that retains repeatedly accessed remote data in the existing requesting-GPM L2, and (1) a paired-demand predictor that identifies adjacent local reads and coarsens HBM reads using HBM row locality. Exact tags, MSHRs, and transaction tables remain authoritative, and a request immediately follows the baseline path when an optimization cannot be applied. CuPath adds no global scheduler or new cache level. Across fourteen workloads, CuPath achieves a 1.492× geometric-mean speedup, deduplicates 9.4% of remote reads, reduces remote request packets by 52.7% through aggregation, and eliminates 29.4% of repeated wafer traversals.**

I. Introduction

Wafer-scale GPUs are emerging as a practical way to continue GPU scaling beyond the size, yield, and manufacturability limits of a monolithic die. MCM-GPUs first address the monolithic-die limits by connecting multiple GPU processing modules (GPMs) through silicon interposers, expanding compute resources and memory capacity within a package [1], [4]. Wafer-scale integration extends the multi-module organization across an entire wafer. Many GPMs and distributed HBM stacks are connected by a high-bandwidth, low-latency on-wafer network and exposed as one logical GPU [13], [25], [34]. Wafer-scale integration is approaching commercial reality, with TSMC's System-on-Wafer technology planned for 2027 [23]. However, placing more resources on one wafer does not make every memory access local.

The on-wafer mesh provides high-bandwidth communication between GPMs, but each physical page still resides in the HBM of one home GPM. A local L1 miss reaches the local L2 cache and, on an L2 miss, accesses local HBM. The returned cacheline fills the local cache hierarchy before reaching the requesting CU.

A remote L1 miss is routed through the requesting GPM's RDMA endpoint, the on-wafer network, and the home GPM's RDMA endpoint and L2 before reaching the home-GPM HBM when necessary. The response returns across the wafer to the requesting GPM's L1, and the baseline does not retain remote returns in the requesting GPM's L2. Duplicate or repeated accesses can therefore pay endpoint, network, cache, and memory costs multiple times. Prior multi-GPU studies show that remote-access cost and sharing behavior vary widely across applications [15], [35].

Local memory exposes a mismatch between regular GPU access patterns and cacheline-by-cacheline processing. Half of the workloads have L2 read miss rates above 80%, allowing related adjacent demands to reach HBM as separate memory requests. Fetching every adjacent line is unsafe because the candidate may already be resident, already be pending, or never be requested. Local acceleration therefore requires a selective decision that learns a recurring relation from real demands and checks current L2 state before adding another cacheline to the HBM access and L2 fill path.

Remote memory remains expensive even with a low-latency on-wafer network. A remote miss still consumes requesting-GPM RDMA service, network bandwidth, home-GPM RDMA service, and home-GPM cache or HBM resources before the response returns. Finite RDMA width and outstanding capacity further make related requests contend for the same endpoint resources. Overall, 42.8% of sampled remote reads expose either an in-flight duplicate or a nearby same-page partner. The baseline then places returned data only in a private requesting-GPM L1, so an access from another CU or after L1 eviction repeats the remote path. Later touches account for 33.6% of remote reads, but retaining every return in the requesting GPM's L2 would waste capacity on one-touch data [36], [37]. Remote acceleration must therefore aggregate requests before network injection and retain returned data in the existing requesting-GPM L2 only after reuse is observed.

We propose *CuPath*, which uses a lightweight demand predictor and a logical typed Cuckoo Filter associated with each GPM's L2 cache to optimize requests during memory-path traversal [11]. The predictor proposes at most one adjacent candidate from real demand history. The Filter records compact pattern, residency, in-flight, and reuse state, allowing later stages to quickly reject work that is already present or unlikely to help. The Filter is a control structure rather than a data cache. Exact tags, MSHRs, and RDMA transaction tables remain authoritative.

CuPath applies the recorded state in three places. First, the

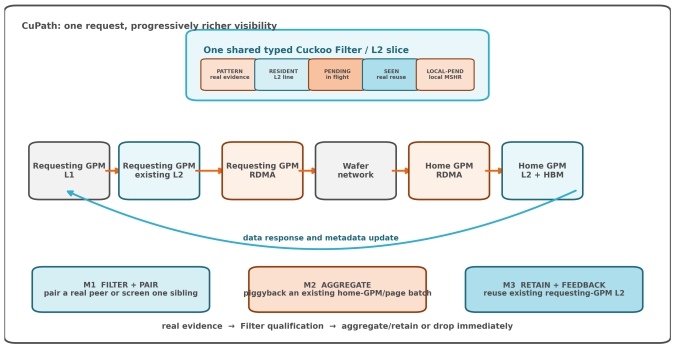


Fig. 1. CuPath continuously filters a predicted request, aggregates the request only with compatible in-flight work, and retains returned data in the existing requesting-GPM L2.

local L2 removes resident or pending candidates and, when the remaining sibling is compatible with the demand, exposes both adjacent cachelines through one paired HBM access. Second, requesting-GPM RDMA uses REMOTE-PENDING to identify possible duplicate lines and REMOTE-BATCH to locate a joinable home-GPM/page batch. Exact RDMA tables confirm the Filter results before merging identical requests or grouping different lines into one packet; a predicted remote line is allowed only when a free position exists in an existing batch. Third, after the response returns, CuPath uses real reuse evidence to retain useful remote data in the existing requesting-GPM L2, avoiding a later wafer traversal. Outcomes feed back to the predictor so that useful relations remain active and unused ones are suppressed.

CuPath is deliberately work-conserving. CuPath never delays a demand while waiting for a partner, never creates a remote packet for a predicted line, and immediately falls back to the original path when metadata or downstream resources are unavailable. CuPath adds no global scheduler or new cache level. Figure 1 summarizes how a request is filtered, combined with useful in-flight work, and selectively retained during memory-path traversal.

Across fourteen GPU workloads, CuPath achieves a 1.492× geometric-mean speedup and improves 13 workloads, while the remaining workload stays within 0.1% of Baseline. More importantly, the speedup follows from work removed along the path. CuPath deduplicates 9.4% of remote reads, reduces remote request packets by 52.7% through aggregation, and eliminates 29.4% of repeated wafer traversals.

We make the following contributions.

- We identify and quantify how predictive locality, cache state, remote aggregation, and reuse become visible at different points in a wafer-scale memory-request lifetime.
- We propose CuPath, an end-to-end memory-path design that couples a bounded real-demand predictor and logical typed Cuckoo Filter with the existing L2 cache, HBM controller, and RDMA engine.
- We implement and evaluate CuPath without adding a global scheduler, cache level, achieving a 1.492× geometric-mean speedup while substantially reducing cache and network work.

II. BACKGROUND

A. Cuckoo Filters

A Cuckoo Filter is an array of buckets, where each bucket contains several slots that are read together [11]. Each occupied slot stores a short fingerprint rather than a complete key. Hashing a key produces its fingerprint and two possible bucket locations. A lookup checks both buckets, while an insertion places the fingerprint in an empty slot or relocates an existing fingerprint when both buckets are full. The structure also supports deletion. A negative lookup proves absence, while a positive lookup remains a possible match and must be checked against exact state.

For example, suppose the address of a cacheline produces fingerprint f and maps to buckets 3 and 11. When the cacheline is filled into L2, the filter stores f in one of the four slots in either bucket. A later L2 data request recomputes the same fingerprint and bucket locations. If neither bucket contains f, the cacheline is not resident and the request can skip the L2 tag and data array lookup before continuing along the miss path. If either bucket contains f, the request checks the exact L2 tag. A tag hit reads the data array, while a false positive simply follows the normal cache miss path. The fingerprint is deleted when the cacheline is evicted. If an insertion fails, the filter conservatively sends later requests to the exact L2 lookup. The filter therefore avoids unnecessary cache probes without changing correctness [24]. It stores only metadata and has finite lookup and update bandwidth.

B. RDMA Request Processing

RDMA moves data between GPMs without involving a host processor in each transfer [14], [22], [33]. Each GPM's RDMA endpoint connects both the private L1 caches and the shared L2 to the wafer mesh. The L1 facing path contains a request table and a pair of TX and RX interfaces. The request table retains every remote memory transaction issued by local L1 caches until its response returns. TX sends these requests to remote GPMs, while RX returns incoming responses to the corresponding L1 requests. The L2 facing path provides another pair of TX and RX interfaces. RX forwards incoming remote requests to the local L2, and TX sends the resulting data responses back to their requesting GPMs. A bounded request table limits the number of requester transactions in flight and propagates backpressure to L1 when no entry is available.

A baseline remote request starts when address translation identifies a remote page. The L1 miss allocates a request-table entry before requester TX injects the packet into the wafer mesh. The owner L2-facing RX accepts the packet and forwards the recovered cacheline request to owner L2. An owner-L2 hit supplies the data directly, while an L2 miss accesses owner HBM. The response returns through owner TX, the wafer mesh, and requester RX. The request table then completes the stored transaction and delivers the data to the original L1 request. In the baseline, same-line misses from different private L1 caches allocate separate entries and network packets before owner L2 can observe and merge them.

C. HBM Organization and Row Locality

HBM provides high bandwidth through a wide interface and many independently operated banks [16], [21], [32], but each bank still follows the conventional DRAM row buffer model [9], [26]. A physical address is decomposed into channel, rank, bank group, bank, row, and column fields. An activate command copies one row into the bank row buffer. A following column read to that open row can transfer data without another activation. A request to a different row must first precharge the bank and then activate the new row. Serving several nearby columns while a row remains open can therefore avoid repeated precharge and activation latency [26]. HBM address mapping and the order in which requests reach the controller determine whether this locality is visible [16], [30], [32]. The simulator models these effects explicitly with transaction and command queues, bank state, burst timing, and separate activate, column, and precharge commands. Its ordinary close page path attaches an automatic precharge to a completed access. When two linked adjacent requests are simultaneously ready and map to the same physical bank and row, the controller changes the first operation into a read without automatic precharge, keeps the row open, and issues the adjacent column next. The row is not retained when the second request is unavailable, maps to a different row, or would bypass an older conflicting request. Adjacent line selection is therefore useful because it creates a bounded opportunity for row reuse, while the controller remains responsible for validating the actual HBM mapping and timing constraints.

III. Observations

We trace the Baseline execution of the same 14 workloads used in our evaluation. O1 quantifies the full memory path. O2 and O3 examine adjacent local reads and their HBM mapping. O4 and O5 examine remote-path cost and requester-side aggregation. O6 examines remote-data reuse and available capacity in the existing requester-GPM L2.

O1: Cost of a Full Memory Path. We measure each component from the arrival of a demand read until the component forwards the request or response. Communication into L1, L2, and HBM is charged to the receiving component. Network Interface contains the time spent at requester-GPM and owner-GPM network interfaces, while Communication contains request and response traversal through the wafer network. We count only completed MSHR leaders and exclude MSHR waiting. Figure 2 shows that HBM and the Local GPM L2 Cache account for 65.8% and 31.0% of the weighted local full-path latency. Network Interface and Communication together contribute 97.0% of the weighted remote full-path latency. Network Interface alone accounts for 80.1%, showing that endpoint contention is more costly than network traversal for the sampled remote paths. Figure 3 further shows that local demand-read miss rates range from 2.8% to 100%, although every miss still pays the full tag-lookup latency. The opportunity therefore includes both avoiding known-miss cache work and reducing downstream memory and communication work.

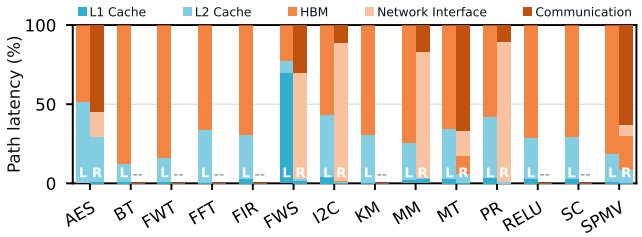


Fig. 2. The full-path data access latency breakdown. Each stacked bar normalizes a completed local (L) or remote (R) MSHR-leader path to 100%. Communication into L1, L2, and HBM is charged to the receiving component, while Network Interface covers the requester-GPM and owner-GPM network interfaces and Communication covers both wafer-network traversals. "–" denotes no qualifying remote path. HBM contributes 65.8% of the weighted local-path latency. Network Interface and Communication contribute 80.1% and 16.9% of the weighted remote-path latency.

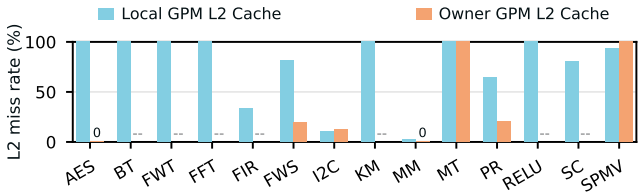


Fig. 3. Demand-read miss rate at the Local GPM L2 Cache and Owner GPM L2 Cache. "–" denotes that no qualifying remote demand path was observed.

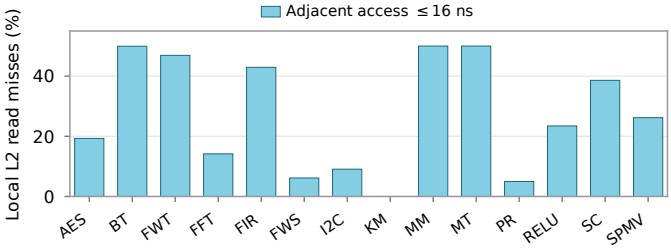


Fig. 4. Short-window adjacent correlation among local L2 read misses. Each bar reports the fraction of selected misses with an earlier miss for the paired adjacent cacheline within 16 ns. Selected requests are completed MSHR leaders that miss in both L1 and L2 and reach local HBM.

O2: Adjacent Local Read Misses Exhibit Short-Window Correlation. Across 13 of 14 workloads, 5.0%–50.0% of local L2 read misses have a preceding read miss for the paired adjacent cacheline within 16 ns (see Fig. 4). We measure the interval between local L2 read misses to the two cachelines in the same aligned HBM region. Together with O3, this result shows that short-window adjacency provides a practical opportunity to exploit HBM row locality through selective paired reads.

O3: Adjacent Reads Preserve HBM Row Locality. We map every physical read to its HBM bank, row, and column. Within 16 controller cycles, 29.1% of reads find an earlier read in the contiguous two-cacheline region of the same HBM row, while only 6.2% find an earlier read at another column of that row, as shown in Fig. 5. The adjacent relation observed at L2 is therefore also the strongest relation exposed by the HBM mapping. A paired read can exploit this locality, provided that

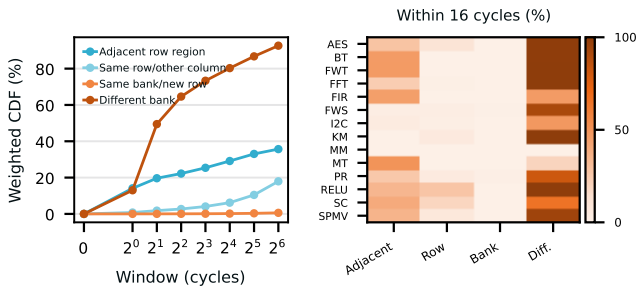


Fig. 5. Physical-read proximity for a contiguous same-row pair, other columns in the same row, a new row in the same bank, and a different bank.

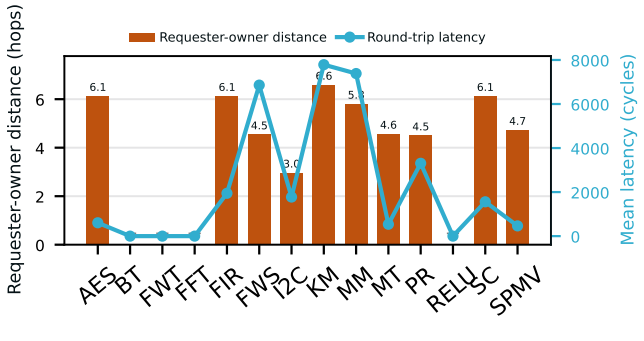


Fig. 6. Remote distance and latency across the wafer. The average distance between requester and owner GPMs ranges from 3.0 to 6.6 mesh hops, while mean end-to-end latency ranges from 465 to 7,787 cycles. "–" denotes no observed remote access.

cache and in-flight state first remove an adjacent line that is already available.

O4: Remote Accesses Amplify Work Across the Wafer. Remote accesses travel an average of 3.0 to 6.6 mesh hops between requester and owner GPMs. Including execution at both RDMA endpoints and the return network path, mean end-to-end remote-access latency ranges from 465 to 7,787 cycles across workloads. Figure 6 reports both measurements. The long path consumes RDMA state and multiple network links before owner L2 can process the request. Owner-L2 coalescing can reduce subsequent memory work, but cannot recover endpoint and network resources already consumed. Requester-GPM RDMA is the first shared component that observes requests from different L1 caches before network injection. These results motivate aggregating remote requests at requester-GPM RDMA so that redundant accesses do not repeat the full wafer traversal.

O5: Remote Requests Can Be Aggregated at Two Granularities. Figure 7 shows two request relations visible at requester-GPM RDMA. Depending on the workload, 0%–91.9% of remote reads duplicate an in-flight request for the same cacheline. After removing those duplicates, another 0%–75.0% of the original remote-read stream targets a different cacheline within the same requester GPM, owner GPM, and 4-KB page during a 16-cycle window. Weighted across all remote reads, the two relations account for 25.9% and 23.1% of the stream. Same-line aggregation avoids a redundant remote access, while same-page aggregation carries different

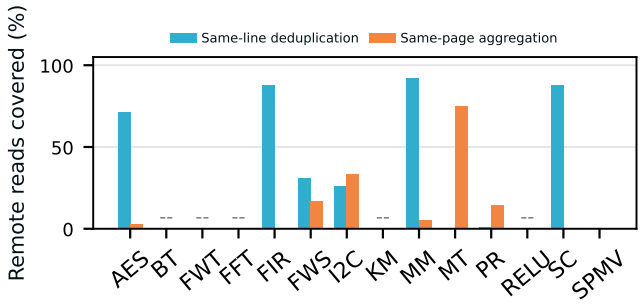


Fig. 7. Remote requests that can be aggregated at requester-GPM RDMA. Same-line requests duplicate an in-flight cacheline access. Same-page requests target different cachelines within the same requester GPM, owner GPM, and 4-KB page during a 16-cycle window. The two relations account for 25.9% and 23.1% of all remote reads, respectively. "–" denotes no observed remote read.

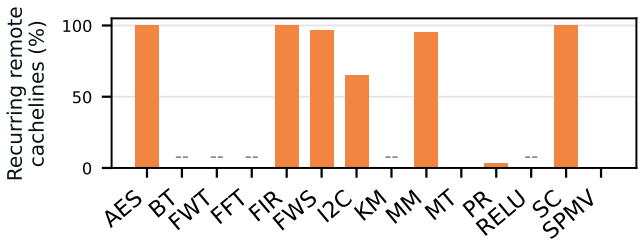


Fig. 8. Fraction of unique remote cachelines accessed more than once. The fraction ranges from 0% to 100% across workloads and is 17.7% when weighted across all remote cachelines. "–" denotes no observed remote read.

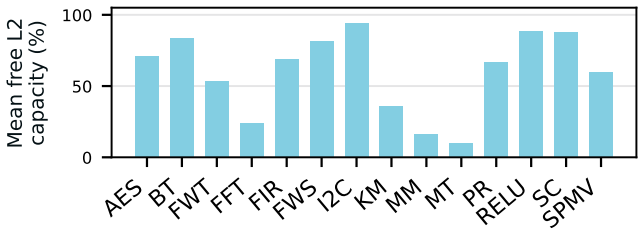


Fig. 9. Mean free capacity in the existing GPM L2 Cache. The average across workloads is 60.1%, with workload means ranging from 9.8% to 94.1%.

cacheline requests in fewer network packets. Processing both relations at requester-GPM RDMA prevents unnecessary traffic before the requests enter the wafer network.

O6: L2 Cache Has Spare Capacity for Reused Remote Data. Depending on the workload, 0%–100% of unique remote cachelines are accessed more than once, as shown in Fig. 8. Weighted across all remote cachelines, 17.7% exhibit reuse, and third-or-later accesses account for 22.0% of the remote-read stream. Figure 9 shows that mean free capacity in the existing GPM L2 Cache ranges from 9.8% to 94.1% across workloads and averages 60.1%. The available L2 Cache capacity can therefore store remote cachelines that have demonstrated reuse. After a second access confirms reuse, the returned cacheline can be installed in the existing L2 Cache, allowing later accesses to complete locally without another wafer traversal or a new cache level.

Together, the observations identify three concrete decision points. Local L2 can reject known-miss tag work and recognize adjacent demand with favorable HBM mapping. Requester-GPM RDMA can remove same-line requests and combine same-page traffic before network injection. Returned remote data can establish reuse before another wafer traversal occurs. CuPath connects these decisions with shared membership state while exact cache, MSHR, and RDMA structures preserve correctness.

IV. Design

CuPath follows a memory request from the local cache hierarchy to HBM and, when necessary, across the wafer. Each stage acts only on information that is already visible at that point. Requester RDMA observes duplicate remote lines and requests headed to the same owner and page. A returned remote line reveals whether the line is likely to be used again. The local L2 observes whether adjacent demand misses repeatedly share one controller access region. CuPath turns these observations into three consecutive transformations shown in Fig. 1. Cuckoo-Filter-Guided Remote Aggregation combines requests before network injection, Cuckoo-Filter-Guided Remote Reuse retains data after recurrence becomes visible, and Cuckoo-Filter-Guided Local Pairing adapts local HBM accesses.

A. Shared Typed Cuckoo Filter

CuPath associates one logical typed Cuckoo Filter with each GPM's existing L2 cache [11]. The Filter provides a shared metadata interface to the L2 and requester RDMA. Both components query and update the same typed membership state, so CuPath does not require a separate RDMA-side Filter.

CuPath extends the conventional fingerprint-only entry with a type flag and a reference count. Each 21-bit slot contains one valid bit, a 13-bit fingerprint, a three-bit type flag, and a four-bit saturating reference count. CuPath forms a lookup key from the process identifier, cacheline-aligned address, metadata type, and owner identifier when needed. Hashing the key produces a fingerprint and two candidate buckets. A slot matches only when both its fingerprint and type match the lookup. The reference count records how many logical insertions share the same fingerprint and type, allowing deletion to clear the slot only after the last reference retires. Adjacent access history and prediction confidence are maintained by the local predictor rather than stored in the Filter.

The type flag identifies the path state represented by each entry. **L2-Resident** tracks cachelines stored in L2, while **L2-In-Flight** tracks cachelines being fetched by an active L2 MSHR. **RDMA-In-Flight** tracks remote requests awaiting responses. **Remote-History** records previously accessed remote cachelines, and **Demand-Pattern** records demand-trained address relationships. All types dynamically share the Filter capacity, while exact remote batches remain in the RDMA request table.

The Cuckoo Filter provides negative membership information rather than authoritative cache or transaction state. L2 fills and evictions maintain **L2-Resident**. A reliable **L2-Resident** negative, together with an exact same-line MSHR check, allows a demand read to bypass the L2 tag pipeline. Requester RDMA maintains **RDMA-In-Flight** while a remote request is active. A reliable negative avoids an unnecessary search of the requesting-GPM RDMA request table.

Possible matches are confirmed by the corresponding exact L2, MSHR, predictor, or RDMA state. History entries receive lower priority so that they cannot displace active state. A Filter lookup costs one cycle and supports 16 parallel lookups per cycle. Duplicate insertions increment the reference count, while deletions decrement it and clear the slot at zero. Unavailable or unreliable lookups follow the exact path, while a correctness-critical **RDMA-In-Flight** lookup retries.

B. Remote Aggregation Guided by the Cuckoo Filter

Motivated by **O4** and **O5**, Remote Aggregation reduces duplicate remote accesses and network packets on the remote memory path. The requester GPM contains the L1 cache that issues the read. The owner GPM stores the target page in local memory. All remote reads from the L1 caches in one requester GPM enter the same RDMA engine. The shared RDMA entry point exposes requests for the same cacheline and requests for different cachelines in the same page of the owner GPM.

For every remote read, the requester GPM RDMA checks the Cuckoo Filter for a slot with a matching fingerprint and the **RDMA In Flight** type. A reliable negative shows that no request for the cacheline is currently recorded. The requester GPM RDMA can therefore create a request table entry without searching existing entries. A possible match triggers an exact request table lookup. When an exact entry already exists, the new L1 read joins the recorded remote access. Only one request reaches the owner GPM, and the returned cacheline serves every L1 read attached to the entry.

After requests for identical cachelines are merged, the requester GPM RDMA groups new request table entries with the same process, owner GPM, and 4 KB page. A bitmap in the request table marks the requested cachelines in each page. A group containing one cacheline uses a normal remote read packet. A group containing multiple cachelines uses one bitmap packet. The requester GPM RDMA sends the group after processing the reads accepted during the current scheduling cycle. The remote predictor may propose another cacheline from the same page. The Cuckoo Filter admits the predicted cacheline only when a learned pattern exists, the cacheline is absent from requester L2, no request for the cacheline is in flight, and a matching page group already exists. The predicted cacheline therefore shares an existing network packet and never creates a separate remote access.

The owner GPM RDMA decodes the bitmap and sends one cacheline read for every marked position to the L2 cache in the owner GPM. An L2 miss continues to the HBM of the owner GPM. The owner GPM RDMA returns each cacheline after the corresponding L2 access completes. The requester GPM RDMA matches each returned cacheline with the request table entry and sends the data to every recorded L1 read. After

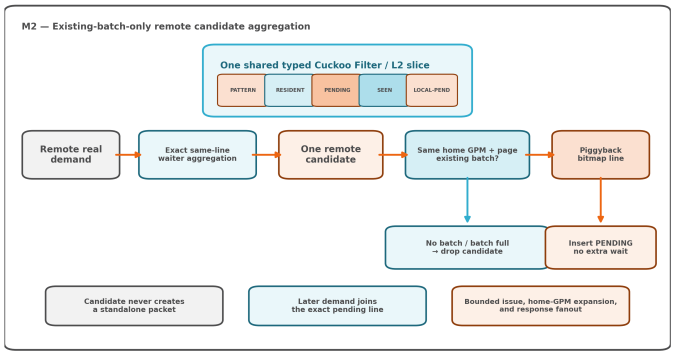


Fig. 10. The RDMA engine in the requester GPM uses the Cuckoo Filter to identify repeated remote accesses and combines the remaining accesses to the same owner GPM and page into one network packet.

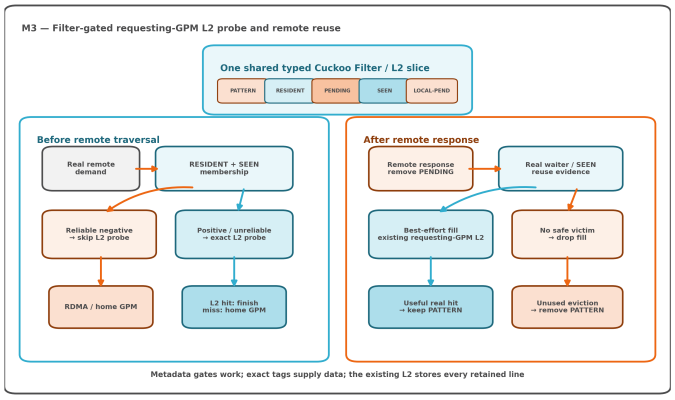


Fig. 11. Remote Reuse records the first remote return and uses later recurrence to retain selected data in the existing L2 cache of the requester GPM.

all recorded reads receive data, the requester GPM RDMA removes the entry and the corresponding **RDMA In Flight** membership.

C. Remote Reuse Guided by the Cuckoo Filter

Motivated by **O6**, Remote Reuse uses the existing requester L2 to avoid repeated remote accesses. Requester RDMA queries the Cuckoo Filter for **L2 Resident** and **Remote History**. Reliable negatives bypass the known miss requester L2 lookup. A first response without additional demand returns to L1 and records **Remote History**. A possible match triggers an exact L2 lookup. A hit completes locally, while a miss follows the remote path.

A returned cacheline becomes eligible for a clean requester L2 fill when **Remote History** records an earlier access, multiple L1 reads join the same remote access, or a real demand claims a predicted cacheline. An unclaimed predicted cacheline without **Remote History** can use only an invalid L2 way. Other eligible fills may use an invalid way or replace a clean remote line, but never an ordinary local line. A successful fill updates **L2 Resident**, allowing later reads to avoid another wafer traversal.

D. Cuckoo-Filter-Guided Local Pairing

Motivated by **O2** and **O3**, Local Pairing reduces L2 tag work and uses HBM row locality for local reads. When a

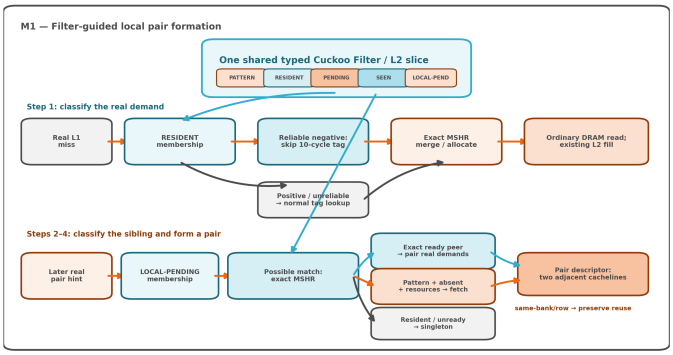


Fig. 12. Local Pairing uses recent demand history to identify adjacent accesses. The typed Cuckoo Filter bypasses proven L2 misses and removes a paired candidate that is already resident or being fetched.

read reaches L2, the typed Cuckoo Filter queries **L2-Resident** while the exact same-line MSHR is checked. A reliable negative and an MSHR miss bypass the L2 tag pipeline, while a possible match retains the exact tag lookup. Local Pairing does not modify writes because the write-back L2 already handles write hits and combines partial writes through its existing MSHRs.

Real L2 misses train a lightweight adjacent-access predictor. The predictor records recently observed cachelines within aligned two-line HBM regions. When real requests access both lines of a region, the predictor increases a saturating confidence counter. Once the confidence reaches its threshold, every subsequent eligible local read miss proposes its adjacent cacheline. Each L2 partition keeps 64 region entries and one two-bit saturating confidence counter. An entry records the process, HBM-controller identity, aligned 128-B region tag, and which of the two cachelines has been observed. Observing the other cacheline retires the matching entry and strengthens confidence. When the table is full, the oldest unmatched entry is replaced and confidence is weakened. The four partitions therefore track up to 256 regions per GPM. The typed Cuckoo Filter then queries **L2-Resident** and **L2-In-Flight** to remove candidates already cached or being fetched. Possible matches are confirmed by the exact L2 tags and MSHRs.

An eligible pair uses one aligned HBM read covering both cachelines within the same row-local access unit [3], [16]. The requested cacheline follows the normal fill path. A request for the adjacent cacheline can join the read before completion. Otherwise, the adjacent data waits in a bounded staging buffer. Later use strengthens the confidence stored in the predictor, while removal without use weakens it. If pairing is unavailable, the original read follows the ordinary cacheline path.

E. End-to-End Composition

The three transformations operate on one request lifecycle. Remote Aggregation uses global request visibility and REMOTE-BATCH state at requester RDMA to remove duplicates and locate compatible network work. Remote Reuse uses completed remote activity to decide whether another wafer traversal can be avoided. Local Pairing uses local demand history and L2 residency to choose the amount of

HBM work. The typed Cuckoo Filter carries compact lifecycle state between these decisions, while exact structures preserve correctness. No mechanism waits for a speculative opportunity. When evidence, metadata bandwidth, or downstream capacity is unavailable, CuPath continues along the baseline path.

V. Evaluation Methodology

A. Simulation

Simulation. We extend MGPUSim [27] with distributed page placement, a packet-switched mesh, RDMA endpoints, detailed cache and HBM timing, and the instrumentation required to model the complete memory path of a wafer-scale GPU. We model 48 GPMs connected by a 7×7 mesh, with the remaining tile hosting the central I/O and control components. Each link provides 768 GB/s of bandwidth and incurs 32 cycles of switch latency, consistent with prior multi-chip and wafer-scale studies [4], [12]. Each GPM represents one quarter of an AMD MI100-class GPU. Four GPMs collectively provide the compute, cache, memory capacity, and memory bandwidth of one MI100 [29]. This scaling keeps cycle-level simulation of the 48 GPM system tractable while preserving its aggregate resource ratios.

Cache and memory hierarchy. Each GPM contains 32 CUs organized as eight shader arrays. Every CU has a private 16 KB, four-way L1 vector cache with a 10-cycle access, 16 MSHRs, and at most 16 concurrent transactions. Each shader array also has a 16 KB scalar cache and a 32 KB instruction cache. The GPM-wide 4 MB L2 is 16-way associative, accepts 64 requests per cycle, contains 256 MSHRs, and charges a 10-cycle directory lookup to both hits and misses. Four memory controllers connect the L2 to 8 GB of local HBM with 1.23 TB/s aggregate bandwidth. Each RDMA endpoint processes eight transfers per cycle, charges 10 cycles for an endpoint traversal, and tracks up to 64 requester and 64 owner transactions. Table I summarizes the modeled system.

CuPath modeling. The MCM-GPU design introduces a GPM-side, remote-only L1.5 cache between L1 and L2 and explores reallocating L2 capacity to that new level [4]. CuPath does not add an L1.5 cache, a victim cache, or another independently addressable cache. Baseline and CuPath therefore use identical L1 and L2 capacities, associativities, ports, MSHRs, and lookup latencies. CuPath adds one logical metadata-only Cuckoo Filter and bounded 64-line M1 response capacity per GPM, together with bounded local and remote prediction state. The response buffer holds an adjacent line returned by a paired HBM access until a matching demand consumes it or replacement removes it. A retained remote response is instead inserted as a clean line through the existing L2 fill and replacement path. Exact cache tags, MSHRs, and RDMA transaction tables remain authoritative. All configurations also use the same TLBs and 4 KB physical page mapping, so every reported difference comes from cache, HBM, RDMA, and network processing below address translation.

Execution scope. Each experiment continues until 78,600 workgroups have completed across the 48-GPM system. This represents approximately 51 completed workgroups for each of the wafer's 1,536 CUs and is sufficient to capture steady execution beyond the initial transient [7]. We terminate at a workgroup boundary because prior studies show that workgroup completion rates stabilize after warmup [5], [20]. The threshold does not change kernel launches, workgroup IDs, partitioning, or GPU assignment. A workload with fewer workgroups runs to completion, and all configurations use the same termination rule.

TABLE I
Baseline wafer-scale GPU and CuPath configuration.

Module	Configuration
Wafer	48 GPMs, 1.0 GHz, 7×7 mesh
Compute	32 CUs/GPM, eight shader arrays/GPM
L1 vector	16 KB/CU, 4-way, 10 cycles, 16 MSHRs
L1 scalar	16 KB/shader array, 4-way, 16 MSHRs
L1 instruction	32 KB/shader array, 4-way, 16 MSHRs
Shared L2	4 MB/GPM, 16-way, 256 MSHRs, 64 req./cycle, 10-cycle lookup
HBM	8 GB/GPM, four controllers, 1.23 TB/s, BL4
RDMA endpoint	8 transfers/cycle, 10-cycle processing, 64 outstanding/direction
Mesh link	768 GB/s, 32-cycle switch latency, 16-B flit
CuPath additions	
Cache capacity	No L1.5 or new addressable cache; baseline L1/L2 unchanged
Typed Filter	Four physical arrays/GPM, 8K buckets/array, four slots/bucket, 13-bit fingerprint
Filter ports	1-cycle lookup/update, 16 parallel lookups and updates/cycle/array
Adjacent-pair predictor	64 region entries/L2 partition (256/GPM), FIFO replacement
Returned-line buffer	16 cachelines/L2 partition (64/GPM), FIFO replacement
Remote predictor	One 256-entry direct-mapped table/requesting GPM
M1	Filter-guided paired HBM access
M2	Up to eight lines/packet, 64 packet descriptors/requesting GPM
M3	No separate history; best-effort clean fill into existing L2

B. Ablation Configurations

We run five configurations with the same benchmark inputs, architectural parameters, workgroup-completion rule, and completed workgroup count. *Baseline* disables every CuPath transformation. *M1* enables the complete local request-pairing path: reliable RESIDENT negatives bypass guaranteed-miss L2 tag lookups, and typed membership state admits at most one predicted sibling into the triggering miss's paired HBM access. *M2* uses REMOTE-PENDING to guide exact same-line deduplication and REMOTE-BATCH to locate joinable, work-conserving home-GPM/page packets. Predicted remote candidates can use only a free position in a Filter-identified and exactly confirmed existing batch; requesting-GPM L2 reuse is off. *M3* enables SEEN/ RESIDENT-guided requesting-GPM L2 probe gating and reuse without remote batching or candidate generation. *Complete* enables M1–M3. Row-aware DRAM reordering remains disabled in all five configurations.

In the complete configuration, the stages operate on the surviving request population. A requesting-GPM L2 termina-

tion never enters requesting-GPM RDMA, and a coalesced RDMA waiter does not create a network or home-GPM memory request. We therefore place counters at each boundary and report both eliminated work and end-to-end performance, rather than treating the three stage speedups as additive.

C. Hardware Overhead

We estimate CuPath's regular-array overhead using CACTI 7.0 at 32 nm [6]. Each GPM contains four physical typed Cuckoo Filter arrays. Each array has 8K buckets, four 21-bit slots per bucket, and 32K slots in total. The four arrays require 336 KiB/GPM. The adjacent-pair predictor contributes 4 KiB/GPM, using 64 entries at a conservative 16 B/entry in each of four L2 partitions. The four returned-line buffers and their metadata contribute another 5 KiB/GPM. The separate 256-entry remote predictor contributes 16 KiB/GPM under a conservative 64 B/entry allocation. We also include the expanded RDMA line and waiter tables, requester and owner batch tables, response-data buffers, and per-L2-line remote-reuse state. All CuPath regular arrays together require 533 KiB/GPM.

We model a Filter as a four-way CACTI cache structure. The 13-bit fingerprint and three-bit type are represented by a 16-bit tag, while the valid and reference-count fields are rounded to one byte. This organization accounts for the Filter storage, tag comparisons, selection logic, banking, and ports. CACTI estimates 3.741 mm^2/GPM for all CuPath regular arrays and 54.510 mm^2/GPM for the complete 4-MiB L2. CuPath therefore adds 6.86% array area relative to L2. The Filter access takes 0.955 ns and fits within the modeled one-cycle lookup at 1 GHz. CACTI does not synthesize hash, arbitration, or control logic, so 6.86% is reported as regular-array area rather than post-layout GPM area.

The completed runs also confirm that the selected Filter capacity is sufficient. Across the fourteen workloads, the mean sum of independently observed per-array peaks is 38.0% of total capacity, while the largest individual Filter reaches 89.9%. The L2-resident, RDMA-in-flight, and paired-access types experience no insertion failure across 214.5 million insertions. Low-priority history entries reject 8.43 million of 40.7 million insertions. Such a rejection only suppresses an optimization hint or restores the exact request path.

D. Workloads and Execution

We evaluate CuPath with 14 benchmarks from Hetero-Mark [28], AMD APP SDK [2], SHOC [8], and DNNMark [10]. The suite has also been used by prior multi-GPU studies [12], [17], [19], [31] and spans local and remote accesses with both streaming and repeated data use. Table II reports these two workload properties, the workgroups actually admitted and retired in the formal run and the workload allocation footprint measured immediately after allocation from the simulator's 4-KB page counters. We derive the labels from the latest Baseline run. **Remote** and **Local** identify whether most memory requests are served by a remote or local home GPM. **Repeated** indicates that L1 and L2 cache hits together serve more than half of read accesses, whereas **Streaming** indicates that most reads pass through both cache levels and continue toward memory. Baseline and Complete allocate the same number of pages and complete the same number of workgroups for every workload.

TABLE II
BENCHMARKS, ACCESS LOCATION, DATA USE, COMPLETED WORKGROUPS, AND FOOTPRINT.

Benchmark	Access	Data use	Completed WGs	Footprint
AES	Local	Repeated	78,600	1,024 MiB
BT	Local	Streaming	78,600	1,024 MiB
FWT	Local	Streaming	78,600	1,024 MiB
FFT	Local	Streaming	78,600	1,024 MiB
FIR	Local	Repeated	78,600	1,024 MiB
FWS	Local	Repeated	78,600	1,024 MiB
I2C	Remote	Repeated	78,600	1,029 MiB
KM	Local	Repeated	73,728	390 MiB
MM	Remote	Repeated	78,600	1,024 MiB
MT	Remote	Streaming	32,400	1,013 MiB
PR	Remote	Repeated	78,600	1,024 MiB
RELU	Local	Streaming	78,600	2,560 MiB
SC	Local	Repeated	78,600	512 MiB
SPMV	Local	Streaming	78,600	1,081 MiB

We choose the largest hundreds-of-MB or GB-scale input that remains tractable in cycle-level simulation, following prior multi-GPU and wafer-scale studies [7], [18], [34]. Every footprint exceeds the capacity of a single GPM's private and shared caches, forcing requests to exercise local HBM and, under distributed page placement, remote home-GPM paths.

The runner records every launch, partition, observed workgroup count, and a hash of the global workgroup set. We reject a comparison if these identities differ across configurations.

VI. RESULTS

A. Overall Performance

Figure 13 compares CuPath with Baseline across the fourteen workloads. CuPath achieves a 1.492× geometric-mean speedup. MM, FWS, and KM obtain the largest gains at 6.180×, 4.341×, and 2.501×, respectively. Local workloads also benefit. BT and FWT improve by 1.635× and 1.645×, while RELU improves by 1.149×. I2C improves by 1.215× through the remote path. Eight workloads improve by more than 1%, while the remaining six stay within 1% of Baseline. SPMV is the only strict regression and remains within 0.1% of Baseline. CuPath therefore provides large gains when the memory path exposes pairing, aggregation, or reuse opportunities without materially penalizing workloads that expose few such opportunities.

B. Ablation Study

Figure 14 isolates the three CuPath transformations. Remote Aggregation is the strongest standalone transformation and achieves a 1.209× geometric-mean speedup. Remote Reuse and Local Pairing achieve 1.156× and 1.108×, respectively. The different geometric means reflect the opportunity exposed at each memory-path stage rather than a uniform benefit across all workloads. Remote Aggregation is most effective when remote reads expose same-line duplication or same-page grouping. FWS and KM consequently reach 2.222× and

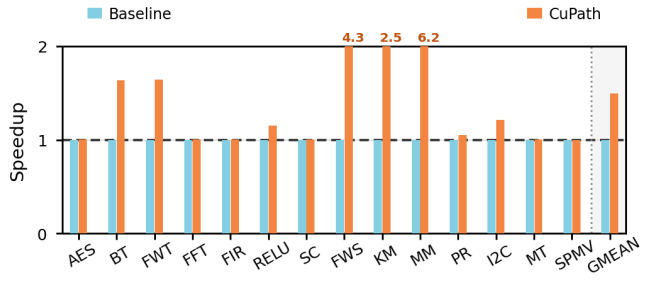


Fig. 13. Performance normalized to Baseline. Bars exceeding 2× are clipped and annotated with their actual speedups. GMEAN is the geometric mean across all fourteen workloads.

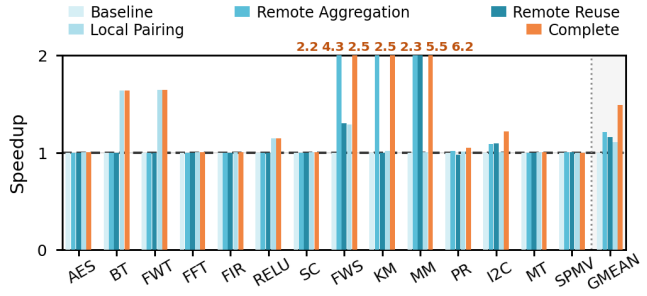


Fig. 14. Ablation of the three CuPath transformations. Remote Aggregation, Remote Reuse, and Local Pairing achieve geometric-mean speedups of 1.209×, 1.156×, and 1.108×, respectively, while Complete reaches 1.492×. Performance is normalized to Baseline. Bars exceeding 2× are clipped, with labels showing the actual speedups.

2.462×. Remote Reuse is more concentrated and provides its largest gain on MM at 5.451×, where later requests can be served by the requesting-GPM L2 instead of crossing the wafer again. I2C also benefits from Remote Reuse at 1.097×.

Local Pairing targets a different request population. BT and FWT improve by 1.635× and 1.645×, and RELU improves by 1.149×. For these workloads, the complete configuration closely follows Local Pairing because little remote traffic reaches the other two transformations. In contrast, workloads that expose fewer usable pairing opportunities remain close to Baseline. The standalone results therefore show that no single transformation explains CuPath's gains across the complete workload set.

Complete CuPath reaches 1.492× and exceeds every standalone configuration in geometric mean. Workloads exposing several opportunities benefit from their composition. FWS improves from 2.222× with Remote Aggregation to 4.341× with Complete. MM combines a 2.341× Remote Aggregation result with a 5.451× Remote Reuse result and reaches 6.180× with Complete. I2C reaches 1.215×, compared with 1.097× for its strongest standalone configuration. When one opportunity dominates, Complete remains close to the corresponding standalone result. KM, for example, reaches 2.501× with Complete and 2.462× with Remote Aggregation. The speedups are complementary but not additive because every stage operates on the requests that remain after earlier stages have removed or combined work.

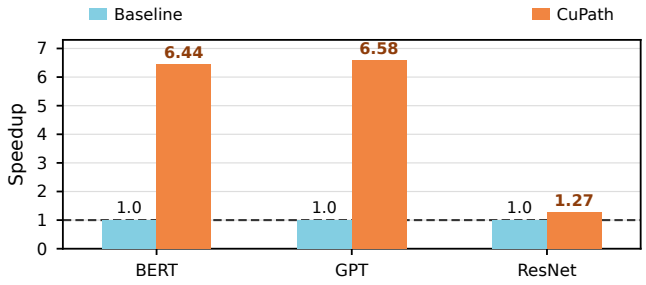


Fig. 15. Normalized performance of CuPath on BERT, GPT, and ResNet. CuPath achieves speedups of 6.44×, 6.58×, and 1.27×, respectively, over the normalized Baseline.

C. Large-Model Workloads

Figure 15 evaluates CuPath on BERT, GPT, and ResNet. CuPath accelerates all three workloads, with substantially larger gains for BERT and GPT than for ResNet. Transformer execution repeatedly exposes adjacent accesses, concurrent remote requests, and remote data reuse, allowing CuPath to remove or combine work at several points along the memory path. ResNet exposes fewer of these opportunities, but still benefits from the same mechanisms. The results show that CuPath extends beyond traditional GPU kernels and remains effective across both transformer and convolutional workloads.

D. Removing Work Along the Path

Figure 16 explains the speedup in terms of work that no longer reaches a downstream component. Same-line deduplication is the number of logical remote reads merged into an existing transaction divided by all logical remote reads. Same-page packet reduction is the difference between post-dedup cacheline transactions and request packets divided by the former. Repeated-traversal elimination counts logical remote reads served by the requesting-GPM L2 Cache, normalized by all logical remote reads. The weighted bars sum the corresponding numerator and denominator across all workloads. Complete deduplicates 9.4% of remote reads, reduces remote request packets by 52.7%, and eliminates 29.4% of repeated wafer traversals.

CuPath does not obtain these gains by increasing aggregate memory traffic. Complete reduces physical DRAM reads from 39.25 to 38.71 million (1.38%) and writes from 20.83 to 20.68 million (0.71%). The sample-weighted local L2-miss-to-DRAM-issue latency falls from 38.12 ns in Baseline to 20.34 ns over the ordinary and Filter-negative Complete issue populations. We report these populations separately in the machine-readable results; the comparison is not an assumption that every miss takes the fast path.

E. Typed Filter Effectiveness

Figure 17 shows how often a negative Filter result eliminates an exact metadata lookup in each CuPath application. *Remote Aggregation* reports remote reads that bypass the requester-GPM RDMA request-table search. *Remote Reuse* reports remote reads that bypass the requester-GPM L2 Cache lookup

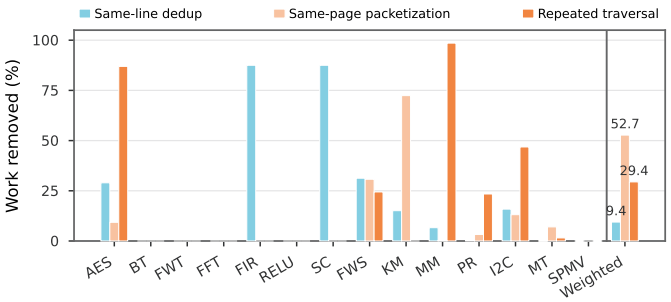


Fig. 16. Remote-path work removed by Complete CuPath. The weighted bars are calculated from aggregate event counts rather than averaging workload percentages.

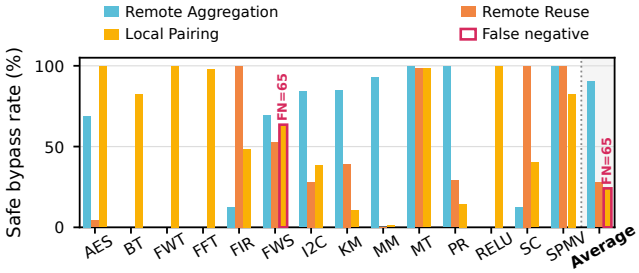


Fig. 17. Exact-path opportunities safely bypassed by the typed Cuckoo Filter. *Average* is calculated from aggregate counts across fourteen workloads and reaches 90.6% for Remote Aggregation, 27.8% for Remote Reuse, and 24.1% for Local Pairing. Missing bars denote no qualifying remote access. Only 65 apparent false negatives occur across all workloads.

because the requested line is not retained there. *Local Pairing* reports local reads that bypass the local-GPM L2 Cache lookup and proceed directly to miss handling. *Average* is calculated from the total bypasses and opportunities across all workloads rather than from the arithmetic mean of workload percentages.

Remote Aggregation safely bypasses 89.57 million requester-RDMA table searches, or 90.6% of all opportunities. The rate exceeds 84% for I2C, KM, MM, MT, PR, and SPMV, showing that most remote reads do not require an exact search before a new transaction is created. Remote Reuse bypasses 13.27 million requester-GPM L2 Cache lookups, corresponding to an aggregate rate of 27.8%. The rate reaches 53.0% for FWS, 98.4% for MT, and 99.8% for SPMV. Missing remote bars indicate that the workload generates no qualifying remote access. Local Pairing bypasses 32.61 million local-GPM L2 Cache lookups, or 24.1% of all opportunities. The rate exceeds 97% for AES, FWT, FFT, and RELU, showing that most Filter-eligible reads in these workloads are absent from L2 Cache. Only 65 apparent false negatives occur across all workloads, all during short L2 Cache write-update windows.

F. Local Pairing Effectiveness

Figure 18 evaluates the requests issued by Local Pairing. Accuracy is the fraction of issued pairs whose adjacent line is later demanded. Coverage is the fraction of local read-miss lines served by an issued adjacent line. Timeliness is the fraction of useful adjacent lines that return to the staging

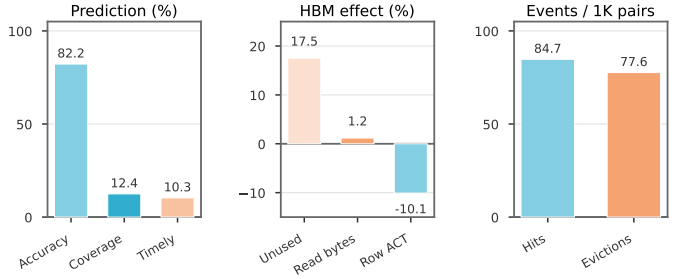


Fig. 18. Aggregate Local Pairing prediction quality, HBM effects, and staging-buffer outcomes. HBM read-byte and row-ACT changes are measured relative to Baseline. Values are calculated from aggregate event counts across all fourteen workloads.

buffer before the demand arrives. Across all workloads, 82.2% of issued pairs are useful and cover 12.4% of local read misses. 10.3% of useful lines are timely, while the remaining useful lines join their paired access in flight.

Classified unused lines account for 17.5% of issued pairs. Their 66.2 MB of data correspond to 2.65% of the demand-miss bytes. After useful paired lines replace later singleton reads, total HBM read traffic increases by 30.0 MB, or 1.17% over Baseline, while aggregate HBM row activations decrease by 10.1%. The staging buffer supplies 500,125 demanded lines and evicts 458,140 unused lines. The normalized bars report both events per 1,000 issued pairs.

G. M2: Cuckoo-Filter-Guided Remote Aggregation

M2 is the largest standalone contributor. Complete records 4.24 million real demands merged by exact RDMA line state and 2.52 million predicted lines piggybacked into existing home-GPM/page packets. Because a real demand can take over a candidate before batch insertion, the aggregate counters do not expose the intersection of these two populations; we therefore do not relabel every useful prediction as an avoided packet. The exact merge and wire-packet counters independently establish the reductions.

H. M3: Cuckoo-Filter-Guided Remote Reuse

M3 bypasses 23.41 million exact requesting-GPM L2 probes when RESIDENT and SEEN reliably prove absence, 42.6% of eligible unique-request probe opportunities. The remaining probes produce 27.23 million physical L2 hits and satisfy 28.21 million logical responses through exact waiter fanout. M3 installs 7.06 million remote-clean lines in the existing L2; 0.996 million are observed to retire unused, a 14.1% lower bound because lines still resident at termination are unclassified. Invalid ways absorb 5.35 million fills and older remote-clean lines absorb the remaining 1.71 million; no ordinary local-clean line is displaced. The largest workload-level sum of peak occupancies reaches 37.5% of wafer L2 lines, a conservative non-simultaneous upper bound rather than an average footprint. Of the unused retirements, 160,257 carry candidate provenance and remove the associated PATTERN. A useful patterned hit instead keeps PATTERN eligible; the aggregate counter does not isolate that subset from all requesting-GPM L2 hits.

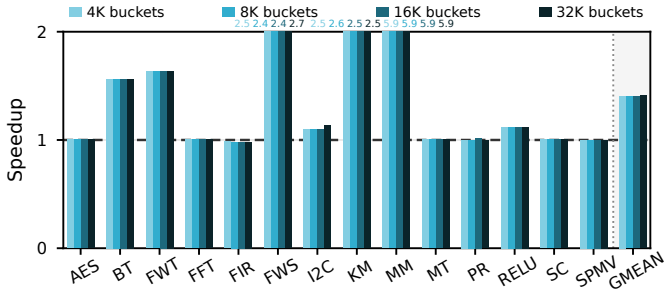


Fig. 19. Sensitivity to aggregate typed Cuckoo Filter capacity per GPM, distributed across four physical arrays and normalized to the fixed overall-performance Baseline. Bars above 2× are clipped.

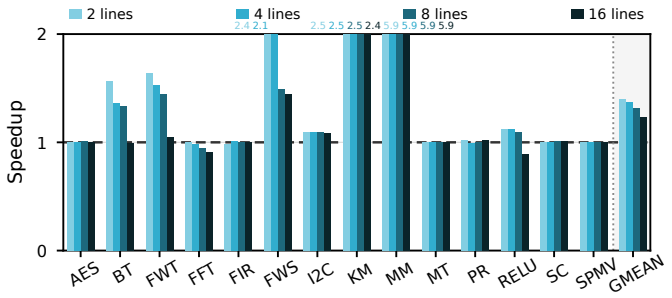


Fig. 20. Sensitivity to the row-local HBM region size, normalized to the fixed overall-performance Baseline. Larger regions fetch more uncertain cachelines and progressively reduce performance.

I. Sensitivity Study

Figures 19–21 evaluate CuPath's sensitivity to the typed Cuckoo Filter capacity, L2 Cache capacity, and the number of cachelines covered by one row-local HBM access. Every Complete configuration is normalized to the same fixed Baseline used by the overall-performance study, and each panel reports all fourteen workloads followed by their geometric mean. Keeping one denominator makes the absolute performance of different configurations directly comparable. For the L2 sweep, this normalization measures end-to-end sensitivity to cache capacity rather than CuPath's incremental benefit over a capacity-matched Baseline.

CuPath is largely insensitive to Filter capacity. Each sweep label gives the aggregate number of buckets/GPM distributed across four physical arrays; the 32K point matches the overall-performance configuration. Increasing capacity from 4K to 32K buckets per GPM produces geometric-mean speedups of 1.474×, 1.471×, 1.470×, and 1.479×. The entire range remains within 0.6% of the 16K-bucket configuration, showing that the Filter does not rely on overprovisioned metadata capacity to retain its performance benefit.

The row-local HBM region has a clearer optimum. Regions containing 2, 4, 8, and 16 cachelines achieve 1.470×, 1.428×, 1.351×, and 1.237×, respectively. A two-cacheline region captures the recurring adjacent access while limiting unnecessary data movement. Expanding the region fetches progressively more lines whose use is less certain and consumes additional HBM and cache resources. The decreasing speedup therefore supports CuPath's adjacent-line design.

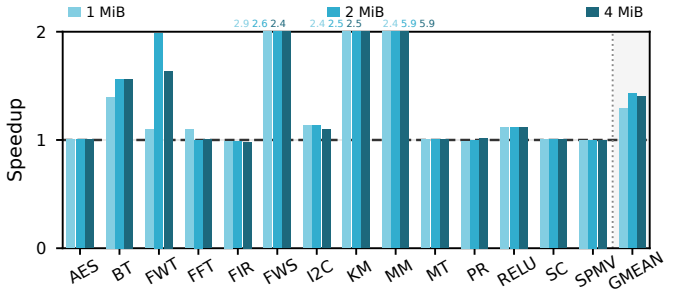


Fig. 21. End-to-end sensitivity to L2 Cache capacity, normalized to the fixed 4-MiB overall-performance Baseline.

L2 capacities of 1, 2, and 4 MiB achieve geometric-mean speedups of 1.249×, 1.448×, and 1.470×, respectively, across all fourteen workloads.

J. Limitations

The long runs expose a limitation hidden by the short screen. Across the fourteen Complete runs, active residency, RDMA, and paired-access state have zero insertion failures. Capacity pressure instead rejects 5.57 million of 13.17 million low-priority remote-history insertions and 2.85 million of 27.59 million demand-pattern insertions. These failures suppress optional reuse or pairing and cannot change demand correctness, but they bound coverage. The modest 1.187× Remote-group speedup and M1's FIR regression likewise show that CuPath is constrained by available aggregation and safe idle opportunities, not by an unlimited speculative upper bound.

A controlled simulator-runtime screen uses seven representative workloads, all five configurations, 192 drained workgroups, one host worker, and the same binary. Normalized by simulated GPU time, Complete costs 1.190× Baseline. This value is simulator overhead rather than hardware performance.

References

[1] Advanced Micro Devices, "Amd accelerates pace of data center ai innovation and leadership with expanded amd instinct gpu roadmap," https://www.amd.com/en/newsroom/press-releases/2024-6-2-amd-accelerates-pace-of-data-center-ai-innovation-.html, 2024, accessed: 2025-04-09.

[2] Advanced Micro Devices, Inc., "Amd opencl programming optimization guide," https://www.amd.com/content/dam/amd/en/documents/radeon-tech-docs/programmer-references/AMD_OpenCL_Programming_Optimization_Guide2.pdf, 2012, accessed: 2025-04-11.

[3] *Random Access Master Attachment (RAMA) LogiCORE IP Product Guide (PG310)*, Advanced Micro Devices, Inc., 2018, version 1.0; 64-B and 128-B burst fragments for the HBM subsystem. [Online]. Available: https://docs.amd.com/api/khub/documents/NrHsi_t5IEEJJcxfgzAX9w/content

[4] A. Arunkumar, E. Bolotin, B. Cho, U. Milic, E. Ebrahimi, O. Villa, A. Jaleel, C.-J. Wu, and D. Nellans, "Mcm-gpu: Multi-chip-module gpus for continued performance scalability," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 320–332, 2017.

[5] C. Avalos Baddouh, M. Khairy, R. N. Green, M. Payer, and T. G. Rogers, "Principal kernel analysis: A tractable methodology to simulate scaled gpu workloads," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 724–737.

[6] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 2, pp. 1–25, 2017.

[7] T. Baruah, Y. Sun, A. T. Dinçer, S. A. Mojumder, J. L. Abellán, Y. Ukidave, A. Joshi, N. Rubin, J. Kim, and D. Kaeli, "Griffin: Hardware-software support for efficient page migration in multi-gpu systems," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 596–609.

[8] A. Danalis, G. Marin, C. McCurdy, J. S. Meredith, P. C. Roth, K. Spafford, V. Tipparaju, and J. S. Vetter, "The scalable heterogeneous computing (shoc) benchmark suite," in *Proceedings of the 3rd workshop on general-purpose computation on graphics processing units*, 2010, pp. 63–74.

[9] W. Ding, J. Liu, M. Kandemir, and M. J. Irwin, "Reshaping cache misses to improve row-buffer locality in multicore systems," in *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 2013, pp. 235–244.

[10] S. Dong and D. Kaeli, "Dnnmark: A deep neural network benchmark suite for gpus," in *Proceedings of the General Purpose GPUs*, 2017, pp. 63–72.

[11] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than bloom," in *Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies*, 2014, pp. 75–88.

[12] Y. Feng, S. Na, H. Kim, and H. Jeon, "Barre chord: Efficient virtual memory translation for multi-chip-module gpus," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 834–847.

[13] Y. Hu, X. Lin, H. Wang, Z. He, X. Yu, J. Zhang, Q. Yang, Z. Xu, S. Guan, J. Fang *et al.*, "Wafer-scale computing: advancements, challenges, and future perspectives [feature]," *IEEE Circuits and Systems Magazine*, vol. 24, no. 1, pp. 52–81, 2024.

[14] A. Kalia, M. Kaminsky, and D. G. Andersen, "Design guidelines for high performance RDMA systems," in *2016 USENIX annual technical conference (USENIX ATC 16)*, 2016, pp. 437–450.

[15] M. Khairy, V. Nikiforov, D. Nellans, and T. G. Rogers, "Locality-centric data and threadblock management for massive gpus," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1022–1036.

[16] B. Li, C. Song, J. Wei, J. H. Ahn, and N. S. Kim, "Exploring new features of high-bandwidth memory for GPUs," *IEICE Electronics Express*, vol. 13, no. 14, p. 20160527, 2016.

[17] B. Li, Y. Guo, Y. Wang, A. Jaleel, J. Yang, and X. Tang, "Idyll: Enhancing page translation in multi-gpus via light weight pte invalidations," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1163–1177.

[18] B. Li, J. Yin, A. Holey, Y. Zhang, J. Yang, and X. Tang, "Trans-fw: Short circuiting page table walk in multi-gpu systems via remote forwarding," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 456–470.

[19] B. Li, J. Yin, Y. Zhang, and X. Tang, "Improving address translation in multi-gpus via sharing and spilling aware tlb design," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1154–1168.

[20] C. Liu, Y. Sun, and T. E. Carlson, "Photon: A fine-grained sampled simulation methodology for gpu workloads," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1227–1241.

[21] Micron Technology, Inc., "Integrating and operating HBM2E memory," Micron Technology, Inc., Technical White Paper, 2021. [Online]. Available: https://assets.micron.com/adobe/assets/urn:aaid:aem:275edf31-79e3-4b6c-8bbd-a233babe9281/renditions/original/as/micron-hbm2e-memory-wp.pdf

[22] R. Mittal, A. Shpiner, A. Panda, E. Zahavi, A. Krishnamurthy, S. Ratnasamy, and S. Shenker, "Revisiting network support for RDMA," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, 2018, pp. 313–326.

[23] S. K. Moore, "Tsmc's advanced packaging: The key to more-than-moore," *IEEE Spectrum*, February 2024, accessed: 2025-06-20. [Online]. Available: https://spectrum.ieee.org/tsmc-advanced-packaging

[24] A. Moshovos, G. Memik, B. Falsafi, and A. Choudhary, "JETTY: Filtering snoops for reduced energy consumption in SMP servers," in *Proceedings HPCA Seventh International Symposium on High-Performance Computer Architecture*. IEEE, 2001, pp. 85–96.

[25] S. Pal, D. Petrisko, M. Tomei, P. Gupta, S. S. Iyer, and R. Kumar, "Architecting waferscale processors-a gpu case study," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 250–263.

[26] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory access scheduling," *ACM SIGARCH Computer Architecture News*, vol. 28, no. 2, pp. 128–138, 2000.

[27] Y. Sun, T. Baruah, S. A. Mojumder, S. Dong, X. Gong, S. Treadway, Y. Bao, S. Hance, C. McCardwell, V. Zhao *et al.*, "Mgpusim: Enabling multi-gpu performance modeling and optimization," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 197–209.

[28] Y. Sun, X. Gong, A. K. Ziabari, L. Yu, X. Li, S. Mukherjee, C. McCardwell, A. Villegas, and D. Kaeli, "Hetero-mark, a benchmark suite for cpu-gpu collaborative computing," in *2016 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2016, pp. 1–10.

[29] TechPowerUp. AMD Radeon Instinct MI100. TechPowerUp. [Online]. Available: https://www.techpowerup.com/gpu-specs/radeon-instinct-mi100.c3496

[30] X. Wang, A. Tumeo, J. D. Leidel, J. Li, and Y. Chen, "MAC: Memory access coalescer for 3d-stacked memory," in *Proceedings of the 48th International Conference on Parallel Processing*, 2019.

[31] Y. Wang, B. Li, M. T. I. Ziad, L. Eeckhout, J. Yang, A. Jaleel, and X. Tang, "Oasis: Object-aware page management for multi-gpu systems." HPCA, 2025.

[32] Z. Wang, H. Huang, J. Zhang, and G. Alonso, "Shuhai: Benchmarking high bandwidth memory on FPGAs," *IEEE Transactions on Computers*, vol. 71, no. 5, pp. 1133–1144, 2022.

[33] X. Wei, Z. Dong, R. Chen, and H. Chen, "Deconstructing RDMA-enabled distributed transactions: Hybrid is better!" in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 233–251.

[34] D. Xu, L. Xu, J. Ren, and Y. Sun, "Exploring the wafer-scale gpus," in *Proceedings of the 17th Workshop on General Purpose Processing Using GPU*, ser. GPGPU '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 8–13. [Online]. Available: https://doi.org/10.1145/3725798.3725800

[35] S. Zhang, M. Naderan-Tahan, M. Jahre, and L. Eeckhout, "Characterizing multi-chip gpu data sharing," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 4, pp. 1–24, 2023.

[36] S. Zhang, M. Naderan-Tahan, M. Jahre, and L. Eeckhout, "SAC: Sharing-aware caching in multi-chip GPUs," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.

[37] X. Zhao, M. Jahre, and L. Eeckhout, "Selective replication in memory-side GPU caches," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 967–980.